RMSNorm Kernel - Part 3 Video Notes

Video: Writing the RMSNorm kernel. This picks up after embedding gather. The code to read alongside it is `rmsNormKernel` and `launchRmsNorm` in `src/kernels.cu`, called from `prefill` in `src/main.cpp`.

The host snippets describe the Part 3 milestone, when prefill stopped after the first norm. The current engine reuses this kernel throughout all 16 layers, the final norm, and decode, with persistent scratch buffers.

What this video is covering

We have token IDs, and embedding gather has turned each ID into a 2,048-value hidden vector. The next operation in the first Transformer layer is RMSNorm. It runs **independently for each token's hidden vector**. If the prompt has `T` tokens, our input and output are both `[T, 2048]`.

```
token IDs [T]
  -> embedding gather -> activations [T, 2048]
  -> RMSNorm with layer 0's input_layernorm weights
  -> normalized activations [T, 2048]
  -> later: Q/K/V projections
```

The word "hidden" just means these numbers are the model's internal representation, rather than text we can read directly. `activations_gpu` contains them already. RMSNorm does **not** look up token IDs or copy the embedding table again. That setup belonged to the previous step.

What are we actually normalizing?

In my working notes I described this as getting the average of the vector close to 1. More precisely, RMSNorm makes the **root mean square magnitude** of the vector close to 1 *before* applying a learned scale. It does not make every value close to 1, and it does not subtract the ordinary mean.

For one hidden vector `x` of width `D = 2048`, with learned weight vector `g` and epsilon `eps = 1e-5`:

```
mean_square = (x[0]^2 + x[1]^2 + ... + x[D-1]^2) / D
inverse_rms = 1 / sqrt(mean_square + eps)
y[j] = x[j] * inverse_rms * g[j]
```

The small `eps` prevents division by zero and keeps very small inputs numerically safe. `g[j]` is one learned weight for hidden column `j`, shared across token positions for this particular normalization layer. Each Transformer layer has **its own** `input_layernorm.weight`, and another weight vector for the norm after attention. There is also a final norm at the end of the full model.

Small example from my notes

Let `x = [1, 2, 3, 4, 5]` and, just for the illustration, all learned weights be `1`. The squared values are `[1, 4, 9, 16, 25]`, their mean is `55 / 5 = 11`, and `sqrt(11)` is about `3.3166`. Ignoring the tiny epsilon, the normalized vector is:

```
[1, 2, 3, 4, 5] / 3.3166
    ≈ [0.302, 0.603, 0.905, 1.206, 1.508]
```

Some elements are still greater than 1. What became 1 is the vector's RMS magnitude. With a nontrivial learned `g`, the output changes column by column. That last multiply is part of RMSNorm, not an optional detail.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

My "brother - man + woman ≈ sister" analogy was meant to give an intuition for a vector's direction, but it is not a rule that this model's actual token embeddings obey. Also, dividing by one scalar preserves direction, while multiplying by a different learned `g[j]` in every dimension can change it. It is better to understand RMSNorm directly from the formula.

Why use RMSNorm here?

In a deep model, activations pass through many layers. Normalizing the size of each token's hidden vector gives the following projections a more controlled input scale. It is one part of how the trained architecture behaves, so we implement it because Llama expects it, not because we can arbitrarily insert it anywhere.

LayerNorm is a related operation: it subtracts the vector's mean and divides by a standard-deviation-like quantity before applying learned parameters. RMSNorm omits the centering step. I said in the rough notes that centering "didn't help"; that is too absolute. The useful distinction for *our code* is simply that RMSNorm needs a **sum of squares**, not a sum of values and a mean subtraction.

Where its inputs come from

In `prefill`, the host has already allocated `activations_gpu` and launched embedding gather. It then allocates a second `[T, 2048]` BF16 buffer, `normalized_gpu`, for the output:

```
const size_t activation_bytes =
    static_cast<size_t>(token_count) * 2048 * sizeof(__nv_bfloat16);

cudaMalloc(&normalized_gpu, activation_bytes);

launchRmsNorm(
    activations_gpu,
    normalized_gpu,
    weights.input_layernorm[0],
    token_count);
```

`weights.input_layernorm[0]` points into the big weight allocation loaded at startup. It is a vector of 2,048 BF16 learned values. We do not copy the weights again for this prompt. The `0` matters: this video works with the **first** layer's input norm. The completed forward pass loops over all 16 layers and selects the corresponding norm vector each time.

The CUDA mapping: one block per token

Here is the interesting parallelism problem. For a single output `y[j]`, we need the sum of **all 2,048 squared input values** for that token. Threads cannot calculate their outputs completely independently as they could in embedding gather. They have to cooperate on that sum.

The launcher uses:

```
rmsNormKernel<<<token_count, 1024>>>(input, output, norm_weights);
```

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

So one CUDA block handles one token. Within the block, 1,024 threads split the 2,048 hidden values. Each thread handles two values, one in each half:

```
const int tid = threadIdx.x;
const int base = blockIdx.x * 2048;

const float x0 = __bfloat162float(input[base + tid]);
const float x1 = __bfloat162float(input[base + tid + 1024]);
```

Example: block 2 handles the third token. Its `base` is `2 * 2048`. Thread 7 reads hidden columns 7 and 1031 of that same token. A different block works on a different token, so it never needs to combine sums with block 2.

The 1,024-thread choice matches this fixed hidden width and the kernel's array size. It is the usual maximum threads per block on the target CUDA hardware, but a device's actual limit should be checked; the code assumes this launch is supported.

Shared memory and the first barrier

Each thread calculates one partial squared sum:

```
__shared__ float partial[1024];

partial[tid] = x0 * x0 + x1 * x1;
__syncthreads();
```

`__shared__` is memory visible to **threads in this block**. All 1,024 entries together represent the sum of squares for all 2,048 values. We convert BF16 inputs to FP32 and keep partial sums in `float` so the reduction loses less precision. The stored model weights and output are still BF16.

`__syncthreads()` is a block-wide barrier: every thread must finish writing its slot before another thread reads a neighbor's slot. I compared it to `pthread_join()` in the rough notes, but a barrier is closer to the truth. The threads keep running after it; we are not ending or joining them.

Tree reduction: adding all the partial sums

If one thread added `partial[0]` through `partial[1023]` on its own, it would leave the other threads idle. Instead, the kernel repeatedly halves the number of useful sums:

```
for (int i = blockDim.x / 2; i > 0; i >>= 1)
{
    if (tid < i)
    {
        partial[tid] += partial[tid + i];
    }
    __syncthreads();
}
```

For the first round, `i = 512`:

```
partial[0]   += partial[512]
partial[1]   += partial[513]
...
partial[511] += partial[1023]
```

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Then 256 threads combine the remaining 512 sums; then 128, 64, and so on down to 1. With eight starting numbers, the mental model is four pair additions, then two, then one: three reduction rounds rather than a seven-addition serial chain. Our kernel takes ten halving rounds from 1,024 partial sums to one total.

The barrier **inside every round** matters. Imagine thread 0 entering the 256 round and reading `partial[256]` before thread 256 finished its work in the 512 round. We would get an incomplete sum. The barrier prevents that. Once the loop ends, the sum of squares for this token is in `partial[0]`.

One thread finishes the scale; all threads write output

The kernel lets thread 0 calculate the inverse RMS and store it back in shared memory:

```
if (tid == 0)
{
    partial[0] = rsqrtf(partial[0] / 2048.0f + 1.0e-5f);
}
__syncthreads();
```

`rsqrtf(z)` computes `1 / sqrt(z)`. That is the same factor we need to divide each input by its RMS. We synchronize once more so every thread reads the finished factor, then thread `tid` writes its two output columns:

```
const float inverse_rms = partial[0];
const float weight0 = __bfloat162float(norm_weights[tid]);
const float weight1 = __bfloat162float(norm_weights[tid + 1024]);

output[base + tid] = __float2bfloat16(x0 * inverse_rms * weight0);
output[base + tid + 1024] =
    __float2bfloat16(x1 * inverse_rms * weight1);
```

Notice the two meanings of "weight" here. `activations_gpu` holds prompt-dependent input values; `norm_weights` holds fixed learned scale parameters. We normalize **activations**, then multiply by those learned parameters.

Launch errors versus execution errors

`launchRmsNorm` returns `cudaGetLastError()` immediately after the kernel launch. That catches an invalid launch configuration or another launch-time failure. GPU work normally runs asynchronously, so it does not by itself prove every thread completed correctly. `prefill` later calls `cudaDeviceSynchronize()` and checks its result to catch execution failures before freeing buffers.

What this code does today, and what comes next

The kernel is reusable for any token count with this fixed 2,048 hidden width. At this video's milestone, `prefill` called it only for layer 0's first norm; Q/K/V projection was the next step. The later Q/K/V notes show how `normalized_gpu` becomes that projection input. Part 7 completes the forward pass and produces the first next-token prediction.

The thing to keep in mind from this video:

```
one block = one token
one thread = two hidden values
shared memory + barriers = one token-wide sum of squares
inverse RMS + learned per-column scale = normalized BF16 output
```

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.